Manual de estudio

Juego de misiones

Preparación del examen práctico de JavaScript (Desarrollo web en entorno cliente, Unidad 3)

Archivos analizados	index.html, juego.html, ayuda.html, script.js, juego.js, ayuda.js, estilos.css, README.md
Proyecto de	Javier Portela - Curso 2026/2027
Cómo usar este manual	1) Lee las secciones 1 a 3 para entender el juego. 2) Practica la sección 4 con el código delante. 3) Haz los ejercicios de la sección 7 sin mirar la solución. 4) Repasa la sección 8 la noche anterior.

Antes de empezar: dos cosas importantes sobre TU código
1) Este juego NO tiene movimiento, enemigos que se muevan, colisiones, vidas, niveles ni temporizadores. Es un juego de misiones por turnos: pulsas Atacar y se genera una puntuación aleatoria. Este manual solo explica lo que existe. Cuando el examen pida 'velocidad', 'vidas' o 'tiempo', la guía de la sección 4 te explica cómo se traduciría a ESTE código.

2) Hay un detalle clave en tu código: la función `guardarRecord()` **existe pero NADIE la llama.** Por eso el récord nunca se guarda. Es el típico 'fallo escondido' de examen: si te piden que funcione el récord, la solución es una sola línea (ejercicio 7 y sección 4).

Índice

1. Explicación general del juego 4

1.1 De qué trata 4

1.2 Mecánicas completas 4

1.3 Qué pasa desde que empieza hasta que termina una partida 4

1.4 Cómo interactúa el jugador 5

1.5 Cómo se conectan los archivos 5

2. Explicación completa del código 6

2.1 index.html (pantalla de perfil) 6

2.2 script.js (lógica de index.html) 6

2.3 juego.html (pantalla del juego) 8

2.4 juego.js (el corazón del juego) 8

2.5 ayuda.html y ayuda.js (ventana adicional) 12

2.6 estilos.css 12

3. Explicación por bloques 14

Bloque A - Variables y constantes 14

Bloque B - Inicio de cada página 14

Bloque C - Validación del perfil 14

Bloque D - Panel del jugador (String, Date, estilos) 15

Bloque E - Ataque y puntuación 15

Bloque F - Victoria 15

Bloque G - Ventana de ayuda 15

Bloque H - Almacenamiento 15

4. Guía para modificar el juego en el examen 16

4.1 Modificaciones fáciles (cambiar un valor) 16

4.2 Cambiar textos y colores 18

4.3 Añadir o quitar cosas (modificaciones medias) 19

4.4 Modificaciones difíciles 21

5. Errores habituales 23

5.1 Errores al modificar el código 23

5.2 Partes que NO se deben tocar sin modificar otras 23

5.3 Errores de sintaxis típicos 24

5.4 Cómo detectar errores en la consola 24

5.5 Cómo comprobar que un cambio funciona 24

6. Diccionario de JavaScript 25

Conceptos que NO aparecen pero conviene conocer 26

7. Ejercicios tipo examen 28

8. Resumen final para estudiar 37

8.1 Esquema general 37

8.2 Funciones más importantes 37

8.3 Variables, claves e ids que debes saber de memoria 37

8.4 Zonas donde se hacen los cambios más habituales 38

8.5 Lista de imprescindibles para aprobar 38

1. Explicación general del juego

1.1 De qué trata

Es un pequeño juego web de **misiones**. El jugador crea un perfil (nombre y fecha de nacimiento), entra al juego y debe conseguir **50 puntos** (constante `OBJETIVO`) atacando a un monstruo. Cada ataque suma o resta puntos al azar. Al llegar a 50 la misión se completa y el botón Atacar se desactiva.

El proyecto sirve para practicar: manipulación del DOM, eventos, `String`, `Date`, `Math`, arrays, ventanas nuevas (`window.open`) y almacenamiento (cookies, `sessionStorage`, `localStorage`).

1.2 Mecánicas completas

Mecánica	Cómo funciona	Dónde está
Validación de acceso	Nombre no vacío, fecha válida, anterior a hoy y edad >= 12. Si falla, 'Acceso denegado'.	`validarPerfil()` en script.js
Recordar jugador	Cookie `ultimoJugador` (7 días) rellena el campo nombre al volver.	script.js: `guardarCookie`, `leerCookie`, DOMContentLoaded
Panel del jugador	Nombre con formato (Primera mayúscula), edad calculada, colores según inicial y edad.	juego.js: `mostrarPanel`, `aplicarEstilosPanel`, `calcularEdad`
Ataque	Número entero aleatorio de -5 a 10. La puntuación nunca baja de 0.	juego.js: `atacar()`
Historial	Array con los puntos de cada ataque, mostrado separado por comas.	`historial` + `join(', ')`
Victoria	Si puntuación >= 50: se calcula mejor ataque y media, se desactiva Atacar.	`terminarMision()`
Derrota	**No existe.** No hay límite de ataques ni de tiempo.	-
Ayuda	Ventana de 400x300 con nombre, puntuación, ataques y pista aleatoria.	juego.js `abrirAyuda`, ayuda.js
Récord	Menos ataques para ganar, guardado en localStorage. Se LEE pero no se GUARDA (fallo).	`mostrarRecord`, `guardarRecord`
Salir	Borra sessionStorage y vuelve al perfil.	`salir()`
Reinicio	**No existe** botón de reinicio. Recargar la página pone puntuación e historial a cero (las variables están en memoria) pero mantiene el perfil (sessionStorage).	-

1.3 Qué pasa desde que empieza hasta que termina una partida

- **Paso 1.** Abres `index.html`. Se carga `script.js` (con `defer`). Al terminar de cargar el HTML (`DOMContentLoaded`) se engancha el evento `click` del botón Entrar y se lee la cookie `ultimoJugador` para rellenar el nombre.
- **Paso 2.** Escribes nombre y fecha y pulsas Entrar: se ejecuta `validarPerfil()`. Si algo falla: 'Acceso denegado' y no se avanza.
- **Paso 3.** Si todo es correcto: se guarda `nombre` y `fechaNacimiento` en **sessionStorage**, se guarda la cookie y se salta a `juego.html` con `window.location.href`.
- **Paso 4.** `juego.js` arranca (`iniciarJuego`): lee el nombre de sessionStorage (si no hay, te devuelve a index.html), pinta saludo, objetivo, récord y panel, y engancha los 3 botones.

- **Paso 5.** Pulsas Atacar varias veces: cada vez `atacar()` genera puntos, actualiza puntuación, historial y mensaje.
- **Paso 6.** Cuando puntuación >= 50 se llama a `terminarMision()`: mensaje final y botón Atacar desactivado.
- **Paso 7.** Pulsas Salir: se limpia sessionStorage y vuelves a index.html. (La cookie y el localStorage NO se borran.)

1.4 Cómo interactúa el jugador

Acción del jugador	Evento	Función que responde
Pulsa Entrar	`click` en #btnEntrar	`validarPerfil`
Pulsa Atacar	`click` en #btnAtacar	`atacar`
Pulsa Ayuda	`click` en #btnAyuda	`abrirAyuda`
Pulsa Salir	`click` en #btnSalir	`salir`
Pulsa Cerrar (ventana ayuda)	`click` en #btnCerrar	`cerrarVentana`

1.5 Cómo se conectan los archivos

```
index.html  --carga-->  css/estilos.css  y  js/script.js
juego.html  --carga-->  css/estilos.css  y  js/juego.js
ayuda.html  --carga-->  css/estilos.css  y  js/ayuda.js

script.js (index)  --sessionStorage: nombre, fechaNacimiento-->  juego.js (juego)
script.js (index)  --cookie: ultimoJugador------------------>   (vuelve a index)
juego.js  --localStorage: record-->  (se conserva entre partidas)
juego.js  --URL ?nombre=..&puntuacion=..&ataques=..-->  ayuda.js (ventana nueva)
```

Cada HTML carga SU propio JavaScript. Los scripts no comparten variables entre páginas: cada página nueva empieza con un JavaScript limpio. Por eso los datos viajan por el almacenamiento del navegador o por la URL.

Mecanismo	Dónde se guarda	Cuánto dura	Se usa para
Cookie	`document.cookie`	7 días (`expires`)	Último jugador (`ultimoJugador`)
sessionStorage	Pestaña actual	Hasta cerrar la pestaña o `clear()`	`nombre`, `fechaNacimiento`
localStorage	Navegador	Para siempre (hasta borrarlo)	`record`
Parámetros URL	En la dirección tras `?`	Solo en esa ventana	Datos hacia ayuda.html

Archivos a tener en cuenta
En el README la estructura prevista solo lista `js/script.js`, pero en tu proyecto real hay tres scripts: `script.js` (index), `juego.js` (juego) y `ayuda.js` (ayuda). Las rutas en los HTML son `css/estilos.css` y `js/...`, así que los .js deben estar dentro de la carpeta `js` y el css dentro de `css`.

2. Explicación completa del código

Todos los elementos importantes, con sus 6 fichas: dónde, para qué, cómo, por qué, qué pasa si se modifica y con qué se relaciona. Ordenado por archivo en el orden en que se ejecutan.

2.1 index.html (pantalla de perfil)

Elemento	Qué es	Quién lo usa
`<script src="js/script.js" defer>`	Carga el JS en el head. `defer` = descargar ya, ejecutar cuando el HTML esté listo.	Requisito 6 del enunciado
`#formPerfil`	Formulario. Lleva botón `type="button"` (no envía).	Solo estructura
`#nombre` (input text)	Campo del nombre	`validarPerfil` lee `.value`; DOMContentLoaded lo rellena con la cookie
`#fechaNacimiento` (input date)	Devuelve texto `AAAA-MM-DD`	`validarPerfil`
`#btnEntrar`	Botón Entrar	Evento click -> `validarPerfil`
`#mensaje`	Párrafo donde sale 'Acceso denegado'	`validarPerfil` (`textContent`)

Por qué `type="button"`
Si el botón fuera `type="submit"`, el formulario se enviaría y la página se recargaría antes de que JavaScript terminara. Con `type="button"` solo se dispara el evento click.

2.2 script.js (lógica de index.html)

const EDAD_MINIMA = 12	
Dónde	script.js, línea 2 (primera línea de código).
Para qué sirve	Edad mínima para poder jugar.
Cómo funciona	Es una constante numérica. `validarPerfil` la compara con la edad calculada.
Por qué se usa	Poner el número en un solo sitio evita 'números mágicos' y permite cambiarlo en un único punto.
Si se modifica o elimina	Cambiar el valor cambia quién puede entrar. Si se borra, `validarPerfil` lanza `ReferenceError` al llegar a la comprobación 4.
Relacionado con	`validarPerfil` (comprobación 4), `calcularEdad`.

document.addEventListener('DOMContentLoaded', function () {...})	
Dónde	script.js, líneas 6-14.
Para qué sirve	Arrancar la página: engancha el botón Entrar y rellena el nombre con la cookie.
Cómo funciona	El navegador lanza `DOMContentLoaded` cuando el HTML está leído. Se pasa una **función anónima** como segundo parámetro. Dentro: `getElementById('btnEntrar')`, `addEventListener('click', validarPerfil)`, `leerCookie('ultimoJugador')` y, si no está vacía (`!== ''`), `value = ultimoJugador`.
Por qué se usa	El script está en el head: sin esperar, los elementos podrían no existir. (Con `defer` ya existirían, pero se deja explícito.)
Si se modifica o elimina	Si borras el `addEventListener`, el botón Entrar no hace nada. Si quitas la parte de la cookie, el nombre no se recuerda.
Relacionado con	`validarPerfil`, `leerCookie`, `#btnEntrar`, `#nombre`.

calcularEdad(fechaNacimiento)	
Dónde	script.js líneas 17-27 (y copia idéntica en juego.js líneas 86-103).
Para qué sirve	Devuelve la edad en años cumplidos a partir de un objeto `Date`.
Cómo funciona	1) `hoy = new Date()`. 2) `edad = año actual - año nacimiento`. 3) `mesDiferencia = mes actual - mes nacimiento` (getMonth va de 0 a 11). 4) Si `mesDiferencia < 0` o (mismo mes y el día de hoy es menor que el del cumpleaños) aún no ha cumplido: `edad--`. 5) `return edad`.
Por qué se usa	Restar solo los años daría un error de 1 año hasta el cumpleaños.
Si se modifica o elimina	Quitar el `edad--` hace que la edad salga 1 año de más hasta el cumpleaños. Si la renombras, hay que cambiar las llamadas.
Relacionado con	`validarPerfil` (la usa en script.js) y `mostrarPanel` (en juego.js). **Está duplicada** porque cada página carga su propio JS.

validarPerfil()	
Dónde	script.js líneas 30-66.
Para qué sirve	Comprueba los datos del formulario y decide si se entra al juego.
Cómo funciona	Lee nombre (`trim()` quita espacios) y fecha (texto). Cuatro filtros con `if ... return`: (1) nombre vacío; (2) fecha vacía o inválida (`isNaN(fecha.getTime())`); (3) fecha >= hoy; (4) edad < EDAD_MINIMA. Cada fallo escribe 'Acceso denegado' y hace `return` (sale de la función). Si pasa todos: `sessionStorage.setItem` x2, `guardarCookie`, y `window.location.href = 'juego.html'`.
Por qué se usa	Patrón de 'cláusulas de guarda': se descartan los casos malos uno a uno y el código bueno queda al final.
Si se modifica o elimina	Quitar un `return` hace que, tras mostrar el error, se siga ejecutando y se entre igualmente. Cambiar un `>=` por `>` permite una fecha igual a ahora.
Relacionado con	`calcularEdad`, `guardarCookie`, `EDAD_MINIMA`, sessionStorage (lo lee juego.js).

guardarCookie(nombre, valor, dias)	
Dónde	script.js líneas 71-83.
Para qué sirve	Crea una cookie con caducidad.
Cómo funciona	Crea `fecha = new Date()`, le suma `dias` con `setDate(getDate() + dias)`, y escribe en `document.cookie` el texto `nombre=valor; expires=FECHA; path=/`. `encodeURIComponent` protege espacios/tildes y `toUTCString()` da el formato de fecha de cookies.
Por qué se usa	Es la forma estándar de crear cookies en JS puro.
Si se modifica o elimina	Si quitas `expires` la cookie muere al cerrar el navegador. Si quitas `path=/` solo valdría para esa carpeta.
Relacionado con	`validarPerfil` (la llama con 7 días), `leerCookie`.

leerCookie(nombre)	
Dónde	script.js líneas 89-107.
Para qué sirve	Devuelve el valor de una cookie o `''` si no existe.
Cómo funciona	`document.cookie.split('; ')` convierte 'a=1; b=2' en `['a=1','b=2']`. Un `for` recorre el array; cada cookie se parte con `split('=')`; si `partes[0] === nombre`, devuelve `decodeURIComponent(partes[1])`. Si termina el bucle sin encontrarla: `return ''`.
Por qué se usa	`document.cookie` no permite pedir una cookie concreta: hay que buscarla a mano.
Si se modifica o elimina	Si devuelve `null` en lugar de `''`, la comprobación `!== ''` de DOMContentLoaded dejaría de funcionar.
Relacionado con	DOMContentLoaded de script.js, `guardarCookie`.

2.3 juego.html (pantalla del juego)

Elemento	Qué muestra / quién lo rellena
`#saludo`	'Jugador: nombre' -> `iniciarJuego`
`#panel` (section)	Contenedor al que `aplicarEstilosPanel` cambia fondo y color
`#nombreFormateado`, `#edad`	Rellenados por `mostrarPanel`
`#objetivo`	Muestra 50 -> `iniciarJuego` (valor de `OBJETIVO`)
`#puntuacion` (empieza en 0)	`atacar`
`#intentos` (empieza en 0)	Número de ataques -> `atacar`
`#record` (empieza en '-')	`mostrarRecord` / `guardarRecord`
`#btnAtacar`, `#btnAyuda`, `#btnSalir`	Botones con click
`#resultado`	Mensaje de cada ataque y mensaje final
`#historial`	Lista de puntos separados por coma

Regla de oro del DOM
Cada `document.getElementById('x')` del JS debe tener un `id="x"` idéntico en el HTML (mayúsculas incluidas). Si añades algo nuevo en pantalla, **primero el HTML con su id** y después el JS que lo usa.

2.4 juego.js (el corazón del juego)

const OBJETIVO = 50	
Dónde	juego.js línea 2.
Para qué sirve	Puntos necesarios para ganar.
Cómo funciona	Constante global. Se muestra en `#objetivo` y se compara en `atacar`.
Por qué se usa	Un solo sitio para cambiar la dificultad.
Si se modifica o elimina	Cambiarla altera la dificultad. **OJO:** la pista de ayuda.js dice 'Necesitas 50 puntos' en texto fijo y no cambiaría sola.
Relacionado con	`iniciarJuego`, `atacar` (condición de victoria), ayuda.js (texto de pista).

let puntuacion = 0; let historial = []	
Dónde	juego.js líneas 5-6.
Para qué sirve	Estado de la partida: puntos actuales y array con el valor de cada ataque.
Cómo funciona	Son variables globales con `let` porque cambian. `puntuacion` se reasigna; `historial` se modifica con `push`. El número de ataques es `historial.length`.
Por qué se usa	Guardar los ataques en un array permite luego calcular media, mejor ataque y mostrar el historial.
Si se modifica o elimina	Si las pones dentro de `atacar`, se reiniciarían a cada clic y el juego nunca acumularía puntos.
Relacionado con	`atacar`, `terminarMision`, `abrirAyuda`.

document.addEventListener('DOMContentLoaded', iniciarJuego)	
Dónde	juego.js línea 8.
Para qué sirve	Lanza `iniciarJuego` cuando el HTML está listo.
Cómo funciona	Pasa el **nombre** de la función (sin paréntesis). Con paréntesis se ejecutaría en el acto, no en el evento.
Por qué se usa	Los botones deben existir antes de engancharles eventos.
Si se modifica o elimina	Quitarlo: no hay panel ni botones activos.
Relacionado con	`iniciarJuego`.

iniciarJuego()	
Dónde	juego.js líneas 10-30.
Para qué sirve	Preparar la pantalla del juego.
Cómo funciona	Lee `nombre` de sessionStorage. Si es `null` (alguien abrió juego.html sin pasar por el perfil) redirige a index.html y hace `return`. Si hay jugador: pone saludo y objetivo, llama a `mostrarRecord()` y `mostrarPanel(nombre)`, y enlaza los tres botones con `addEventListener('click', ...)`.
Por qué se usa	Protege la página: no se puede jugar sin perfil válido.
Si se modifica o elimina	Si quitas el `return` tras redirigir, seguiría ejecutando código sin nombre y fallaría. Si quitas una línea `addEventListener`, ese botón queda sin vida.
Relacionado con	script.js (guarda `nombre`), `mostrarRecord`, `mostrarPanel`, `atacar`, `salir`, `abrirAyuda`.

atacar()	
Dónde	juego.js líneas 32-60.
Para qué sirve	Es la mecánica principal: un turno de ataque.
Cómo funciona	1) `Math.random()*16` da un decimal de 0 a 15.99; `Math.floor` lo redondea a entero 0-15; `- 5` lo pasa a -5..10. 2) `Math.max(0, puntuacion + puntos)` impide bajar de 0. 3) `historial.push(puntos)`. 4) Actualiza `#puntuacion`, `#intentos` (`historial.length`) e `#historial` (`join(', ')`). 5) Mensaje con if/else if/else según sea positivo, negativo o 0. 6) Si `puntuacion >= OBJETIVO`: `terminarMision()`.
Por qué se usa	Concentra aleatoriedad, redondeo, array, y condiciones: justo lo que pide el enunciado.
Si se modifica o elimina	Cambiar `16` y `5` cambia el rango. Quitar `Math.max` permite puntuaciones negativas. Quitar `push` rompe historial y número de ataques.
Relacionado con	`puntuacion`, `historial`, `OBJETIVO`, `terminarMision`, todos los span del HTML.

terminarMision()	
Dónde	juego.js líneas 62-75.
Para qué sirve	Cerrar la partida ganada.
Cómo funciona	Un `for` recorre `historial` sumando en `suma`. `Math.round(suma / historial.length)` = media redondeada. `Math.max(...historial)` = mejor ataque (el `...` esparce el array como argumentos). Desactiva `#btnAtacar` (`disabled = true`) y escribe el mensaje final.
Por qué se usa	Da el resumen y evita seguir atacando tras ganar.
Si se modifica o elimina	Quitar `disabled = true` permite seguir atacando y volver a lanzar el mensaje. **Aquí falta llamar a `guardarRecord(historial.length)`.**
Relacionado con	`historial`, `#btnAtacar`, `#resultado`, `guardarRecord`.

salir()	
Dónde	juego.js líneas 77-81.
Para qué sirve	Cerrar sesión.
Cómo funciona	`sessionStorage.clear()` borra todo lo de la sesión y `window.location.href = 'index.html'` navega al perfil.
Por qué se usa	Sin datos de sesión, volver atrás a juego.html te echaría de nuevo a index.
Si se modifica o elimina	Si quitas `clear()`, el jugador sigue 'logueado'. Cookie y récord se mantienen siempre.
Relacionado con	`iniciarJuego` (comprueba el null), script.js.

calcularEdad(fechaNacimiento)	
Dónde	juego.js líneas 86-103.
Para qué sirve	Copia exacta de la de script.js (ver 2.2).
Cómo funciona	Igual que arriba.
Por qué se usa	Las páginas no comparten JS; habría que duplicarla o crear un archivo común.
Si se modifica o elimina	Misma consecuencia.
Relacionado con	`mostrarPanel`.

mostrarPanel(nombre)	
Dónde	juego.js líneas 108-125.
Para qué sirve	Rellena el panel con nombre formateado y edad.
Cómo funciona	`nombre.charAt(0).toUpperCase()` + `nombre.slice(1).toLowerCase()` -> 'jAVIER' pasa a 'Javier'. Recupera `fechaNacimiento` (texto) de sessionStorage, lo convierte con `new Date(texto)` y llama a `calcularEdad`. Escribe ambos con `textContent` y llama a `aplicarEstilosPanel`.
Por qué se usa	Cumple el requisito de usar métodos de String y de Date.
Si se modifica o elimina	Si quitas `slice(1)` el nombre queda en una sola letra. Si cambias `.toLowerCase()`, el resto del nombre conservaría sus mayúsculas.
Relacionado con	`calcularEdad`, `aplicarEstilosPanel`, `#nombreFormateado`, `#edad`.

aplicarEstilosPanel(nombre, edad)	
Dónde	juego.js líneas 132-158.
Para qué sirve	Cambia colores del panel según los datos.
Cómo funciona	Guarda el panel en `panel`. `inicial = nombre.charAt(0).toLowerCase()`. `'aeiouáéíóú'.includes(inicial)`: si es vocal, fondo `#1f6f5c` (verde); si no, `#7a3e8e` (morado). Si `edad < 18`, texto `#ffe066` (amarillo); si no, `#ffffff`.
Por qué se usa	Cumple el requisito de 'cambios de estilo según los datos'.
Si se modifica o elimina	`panel.style.xxx` (inline) gana sobre `estilos.css`. Si quitas esta llamada se ven los colores por defecto de `#panel` en el CSS.
Relacionado con	`mostrarPanel`, `estilos.css` (`#panel`).

abrirAyuda()	
Dónde	juego.js líneas 163-178.
Para qué sirve	Abrir la ventana adicional pasándole datos.
Cómo funciona	Lee `nombre`. Construye `url = 'ayuda.html?nombre=..&puntuacion=..&ataques=..'` (usa `encodeURIComponent` en el nombre). `window.open(url, 'ayuda', 'width=400,height=300')` abre la ventana.
Por qué se usa	Sirve para pasar datos entre páginas sin servidor.
Si se modifica o elimina	Cambiar 'width=400,height=300' cambia el tamaño. Si cambias un nombre de parámetro (por ej. `puntuacion`), hay que cambiarlo también en ayuda.js (`parametros.get`).
Relacionado con	`puntuacion`, `historial`, ayuda.js, ayuda.html.

mostrarRecord()	
Dónde	juego.js líneas 182-190.
Para qué sirve	Mostrar el récord guardado al entrar.
Cómo funciona	`localStorage.getItem('record')` devuelve texto o `null`. Si no es `null`, lo escribe en `#record`.
Por qué se usa	El récord debe sobrevivir entre partidas y sesiones: localStorage.
Si se modifica o elimina	Si lo borras, el récord siempre muestra '-'.
Relacionado con	`iniciarJuego`, `guardarRecord`.

guardarRecord(ataques)	
Dónde	juego.js líneas 196-206.
Para qué sirve	Guardar el récord si es el primero o si mejora el anterior.
Cómo funciona	Lee `recordActual`. Si es `null` o `ataques < Number(recordActual)` hace `localStorage.setItem('record', ataques)` y actualiza `#record`. `Number()` es necesario porque localStorage guarda texto ('9' < '10' sería false al comparar como texto).
Por qué se usa	Récord = MENOS ataques para ganar.
Si se modifica o elimina	**Nunca se llama desde ningún sitio**: el récord no se guarda. Se arregla con `guardarRecord(historial.length);` dentro de `terminarMision`.
Relacionado con	`mostrarRecord`, `terminarMision` (debería llamarla), `#record`.

2.5 ayuda.html y ayuda.js (ventana adicional)

const pistas = [...]	
Dónde	ayuda.js líneas 8-13.
Para qué sirve	Array de 4 textos con pistas.
Cómo funciona	Array de strings; se elige una al azar.
Por qué se usa	Un array permite añadir pistas sin tocar la lógica.
Si se modifica o elimina	Añadir un elemento lo hace elegible automáticamente (usa `pistas.length`). Los textos '50' y '-5/10' están escritos a mano.
Relacionado con	`mostrarAyuda`.

mostrarAyuda()	
Dónde	ayuda.js líneas 18-37.
Para qué sirve	Pintar los datos recibidos y una pista.
Cómo funciona	`new URLSearchParams(window.location.search)` lee los parámetros de la URL. Con `get('nombre')` etc. los pone en los span; si no existen `get` devuelve `null` y `|| '-'` pone un guion. `posicion = Math.floor(Math.random()*pistas.length)` elige la pista. Enlaza el botón Cerrar.
Por qué se usa	Recibir datos del juego en la ventana nueva.
Si se modifica o elimina	Nombres de parámetros distintos entre juego.js y ayuda.js = guiones '-' en la ventana.
Relacionado con	`abrirAyuda` (juego.js), `ayuda.html` (ids `nombre`, `puntuacion`, `ataques`, `pista`).

cerrarVentana()	
Dónde	ayuda.js líneas 39-42.
Para qué sirve	Cierra la ventana.
Cómo funciona	`window.close()`. Solo funciona porque la ventana la abrió un script con `window.open`.
Por qué se usa	Cierre desde el botón.
Si se modifica o elimina	Si abres ayuda.html a mano, el navegador puede ignorar `close()`.
Relacionado con	`#btnCerrar`.

2.6 estilos.css

Selector	Qué hace
`* { box-sizing: border-box }`	El padding y borde cuentan dentro del ancho.
`body`	Flexbox para centrar `main`; fondo #1e1e2f, texto blanco.
`main`	Tarjeta de 380px máx., fondo #2b2b44, esquinas redondeadas y sombra.
`h1`	Título centrado, 1.5rem.
`form`	Columna (`flex-direction: column`) con separación de 8px.
`label`, `input`	Etiquetas en negrita; inputs con padding y sin borde.
`button`, `button:hover`	Botón violeta #6c63ff, cambia a #5148d8 al pasar el ratón.
`#mensaje`	Centrado, negrita, altura mínima 1.2em (para que no salte la página al aparecer el texto).

Selector	Qué hace
`#panel`, `#panel h2`	Estilo base del panel. El JS pisa `background-color` y `color` con estilos en línea.

Los tres HTML comparten este único CSS: un cambio aquí afecta a las tres pantallas.

3. Explicación por bloques

Se adaptan los apartados a lo que tu código tiene realmente. Los que no existen se indican como tal.

Bloque típico	¿Existe?	Equivalente en tu código
Variables y constantes	Sí	`EDAD_MINIMA`, `OBJETIVO`, `puntuacion`, `historial`, `pistas`
Inicio del juego	Sí	`iniciarJuego`, DOMContentLoaded
Movimiento del jugador	No	-
Controles y eventos	Sí (solo ratón)	4 eventos `click`. No hay teclado.
Enemigos y obstáculos	Parcial	Solo 'el monstruo', que no tiene código propio (es un texto en el HTML)
Colisiones	No	-
Puntuación	Sí	`atacar`, `puntuacion`, récord
Vidas y niveles	No	-
Temporizadores	No	No hay `setInterval` ni `setTimeout`
Victoria y derrota	Solo victoria	`terminarMision`
Reinicio	No	Solo `salir` o recargar
Interfaz gráfica	Sí	HTML + estilos.css + `mostrarPanel`/`aplicarEstilosPanel`

Bloque A - Variables y constantes

```
const EDAD_MINIMA = 12;          // script.js
const OBJETIVO = 50;             // juego.js
let puntuacion = 0;              // juego.js
let historial = [];              // juego.js
const pistas = [ ... ];          // ayuda.js
```

`const` para lo que NO se reasigna, `let` para lo que cambia. `historial` es `let` aunque se le haga `push` (podría ser `const` porque el array no se reasigna; funciona igual).

Bloque B - Inicio de cada página

Las tres páginas arrancan con `DOMContentLoaded`. Es siempre el mismo patrón: 1) esperar al DOM, 2) leer datos, 3) pintar, 4) enganchar botones.

```
// index:  DOMContentLoaded -> engancha btnEntrar + rellena nombre con cookie
// juego:  DOMContentLoaded -> iniciarJuego (comprueba sesión, pinta, engancha 3 botones)
// ayuda:  DOMContentLoaded -> mostrarAyuda (lee URL, pinta, engancha cerrar)
```

Bloque C - Validación del perfil

Cuatro filtros consecutivos en `validarPerfil`. Todos muestran el mismo mensaje; si te piden mensajes distintos, solo cambias el texto de cada `if` (ejercicio 6).

Filtro	Condición para DENEGAR	Método usado
1	`nombre === ''`	`trim()`
2	`fechaTexto === '' || isNaN(fecha.getTime())`	`Date`, `getTime`, `isNaN`
3	`fecha >= new Date()`	Comparar fechas
4	`calcularEdad(fecha) < EDAD_MINIMA`	`calcularEdad`

Bloque D - Panel del jugador (String, Date, estilos)

- `charAt`, `slice`, `toUpperCase`, `toLowerCase` -> nombre formateado.
- `new Date(texto)` + `getFullYear/getMonth/getDate` -> edad.
- `includes` + `style.backgroundColor` / `style.color` -> colores según inicial y edad.

Bloque E - Ataque y puntuación

```
const puntos = Math.floor(Math.random() * 16) - 5;   // -5 .. 10
puntuacion = Math.max(0, puntuacion + puntos);        // nunca < 0
historial.push(puntos);                               // array
```

Valor de `puntos`	Mensaje	Efecto
> 0 (1 a 10)	'¡Golpe! +N puntos'	Suma
< 0 (-5 a -1)	'Fallaste: -N puntos'	Resta (hasta mínimo 0)
= 0	'El ataque no tuvo efecto'	Nada

Probabilidad: hay 16 valores posibles (-5...10), cada uno 1/16. Valor medio esperado = 2.5 puntos por ataque, así que se necesitan unos 20 ataques de media.

Bloque F - Victoria

Solo se comprueba al final de cada `atacar`: `if (puntuacion >= OBJETIVO) terminarMision();` Se usa `>=` (y no `===`) porque se puede pasar de 50 (por ejemplo 48 + 9 = 57).

Bloque G - Ventana de ayuda

```
juego.js:  window.open("ayuda.html?nombre=Javier&puntuacion=20&ataques=4", "ayuda", "width=400,height=300")
ayuda.js:  new URLSearchParams(window.location.search).get("puntuacion")   // "20"
```

Los parámetros siempre llegan como **texto**. Para hacer cuentas con ellos hay que usar `Number()`.

Bloque H - Almacenamiento

```
// Cookie          guardarCookie("ultimoJugador", nombre, 7)    / leerCookie("ultimoJugador")
// sessionStorage  setItem("nombre", ..)  setItem("fechaNacimiento", ..)  getItem  clear
// localStorage    getItem("record")      setItem("record", ataques)
```

Regla fija de storage
`setItem(clave, valor)` guarda; `getItem(clave)` lee (devuelve `null` si no existe y SIEMPRE texto); `removeItem(clave)` borra una; `clear()` borra todas. localStorage y sessionStorage tienen la misma API.

4. Guía para modificar el juego en el examen

Como tu juego no tiene velocidad, vidas ni tiempo, aquí tienes primero una **tabla de traducción**: qué suele pedir un examen y qué parte de TU código toca.

Si el examen pide...	En tu juego significa...	Dónde
Cambiar la dificultad / velocidad	Cambiar `OBJETIVO` o el rango de puntos del ataque	juego.js: línea 2 y `atacar`
Añadir o quitar vidas	Crear variable `vidas` y bajarla al fallar; o límite de ataques	`atacar`, HTML
Cambiar la puntuación	Fórmula `Math.floor(Math.random()*16) - 5`	`atacar`
Cambiar tiempo de partida	No hay temporizador: se añadiría `setTimeout`/`setInterval`	iniciarJuego (ejercicio 17)
Modificar victoria/derrota	Condición `puntuacion >= OBJETIVO` y `terminarMision`	`atacar`, `terminarMision`
Eliminar elementos	Quitar el HTML y TODAS las líneas JS que usan ese id	HTML + JS
Cambiar controles	Añadir/cambiar eventos (`keydown`, `dblclick`...)	`iniciarJuego`
Textos, colores	Textos en `textContent`, colores en `aplicarEstilosPanel` y CSS	juego.js, estilos.css
Nueva funcionalidad	Nueva función + id en HTML + evento en `iniciarJuego`	Varios
Datos que persisten	Cookie, sessionStorage o localStorage	script.js / juego.js
Cambiar la ventana de ayuda	Parámetros de la URL y `window.open`	`abrirAyuda`, ayuda.js

Método de 4 pasos para CUALQUIER cambio
1) Localiza qué **pantalla** es (index, juego o ayuda) y por tanto qué .js. 2) Busca (Ctrl+F) el id, la variable o el texto que ves en pantalla. 3) Si añades algo visible: HTML primero (con id), JS después, CSS si procede. 4) Recarga con Ctrl+F5 y mira la consola (F12).

4.1 Modificaciones fáciles (cambiar un valor)

Cambiar los puntos necesarios para ganar

Qué buscar: La constante `OBJETIVO` (juego.js, línea 2).

ANTES

```
const OBJETIVO = 50;
```

DESPUÉS

```
const OBJETIVO = 100;
```

Por qué funciona: El valor se muestra en pantalla con `iniciarJuego` y se compara en `atacar`, así que ambos se actualizan solos.

Ojo: El texto de ayuda.js 'Necesitas 50 puntos...' no cambia solo. Si quieres coherencia, edita también esa pista.

Cambiar la edad mínima

Qué buscar: `EDAD_MINIMA` (script.js, línea 2).

ANTES

```
const EDAD_MINIMA = 12;
```

DESPUÉS

```
const EDAD_MINIMA = 18;
```

Por qué funciona: `validarPerfil` compara `calcularEdad(fecha) < EDAD_MINIMA`; con 18, los menores reciben 'Acceso denegado'.

Cambiar el rango de puntos de cada ataque (la 'fuerza')

Qué buscar: `atacar`, línea `const puntos = ...`.

ANTES

```
const puntos = Math.floor(Math.random() * 16) - 5;   // -5 .. 10
```

DESPUÉS

```
const puntos = Math.floor(Math.random() * 21);       // 0 .. 20
```

Por qué funciona: `Math.random()*N` + `floor` da enteros 0..N-1. Restando K se desplaza el rango a (-K .. N-1-K).

Ojo: Para -10..+20 son 31 valores: `Math.floor(Math.random() * 31) - 10`. Actualiza también el comentario y la pista 'entre -5 y 10' de ayuda.js.

Que la puntuación SÍ pueda ser negativa (quitar el suelo de 0)

Qué buscar: `atacar`, la línea con `Math.max`.

ANTES

```
puntuacion = Math.max(0, puntuacion + puntos);
```

DESPUÉS

```
puntuacion = puntuacion + puntos;   // o: puntuacion += puntos;
```

Por qué funciona: `Math.max(0, x)` devolvía el mayor de 0 y x, es decir, nunca bajaba de 0. Sin él, la suma es directa.

Cambiar la ventana de ayuda (tamaño y nombre)

Qué buscar: `abrirAyuda`, línea `window.open`.

ANTES

```
window.open(url, "ayuda", "width=400,height=300");
```

DESPUÉS

```
window.open(url, "ayuda", "width=600,height=450");
```

Por qué funciona: El tercer parámetro es una cadena de características separadas por comas.

Duración de la cookie

Qué buscar: script.js, en `validarPerfil`: `guardarCookie('ultimoJugador', nombre, 7)`.

ANTES

```
guardarCookie("ultimoJugador", nombre, 7);
```

DESPUÉS

```
guardarCookie("ultimoJugador", nombre, 30);
```

Por qué funciona: El tercer parámetro son los días que se suman a la fecha en `guardarCookie`.

4.2 Cambiar textos y colores

Cambiar los colores del panel

Qué buscar: `aplicarEstilosPanel` (juego.js).

ANTES

```
panel.style.backgroundColor = "#1f6f5c";   // vocal
...
if (edad < 18) { panel.style.color = "#ffe066"; }
```

DESPUÉS

```
panel.style.backgroundColor = "#b22222";   // vocal -> rojo
...
if (edad < 21) { panel.style.color = "#00ffff"; }
```

Por qué funciona: Cada color es un texto hexadecimal asignado a la propiedad `style`. Cambiar el 18 cambia el límite de edad del color.

Ojo: Una propiedad CSS con guion se escribe en camelCase en JS: `background-color` -> `backgroundColor`.

Cambiar los mensajes del ataque

Qué buscar: `atacar`, bloque `if (puntos > 0) ... else if ... else`.

ANTES

```
resultado.textContent = "¡Golpe! +" + puntos + " puntos";
```

DESPUÉS

```
resultado.textContent = "¡Crítico! Has hecho " + puntos + " de daño";
```

Por qué funciona: Solo es texto concatenado con `+` a una variable.

Cambiar el texto de acceso denegado

Qué buscar: script.js, `validarPerfil`: aparece 4 veces.

ANTES

```
mensaje.textContent = "Acceso denegado";
```

DESPUÉS

```
mensaje.textContent = "Debes tener al menos " + EDAD_MINIMA + " años";   // solo en el filtro 4
```

Por qué funciona: Si quieres cambiarlo en los 4 sitios hazlo en los 4; si solo en uno, solo ese. Usar `EDAD_MINIMA` evita que el texto se desfase.

4.3 Añadir o quitar cosas (modificaciones medias)

Arreglar/activar el récord (la modificación más probable)

Qué buscar: `terminarMision` (juego.js). `guardarRecord` ya existe y está sin usar.

ANTES

```
function terminarMision() {
    ...
    document.getElementById("btnAtacar").disabled = true;
    document.getElementById("resultado").textContent = ...;
}
```

DESPUÉS

```
function terminarMision() {
    ...
    document.getElementById("btnAtacar").disabled = true;
    document.getElementById("resultado").textContent = ...;
    guardarRecord(historial.length);   // <-- nueva línea
}
```

Por qué funciona: `historial.length` es el número de ataques de la partida. `guardarRecord` guarda si es el primero o menor que el anterior y actualiza `#record`.

Ojo: Para probarlo: gana una partida, recarga la página y comprueba que el récord sigue. Para borrar el récord al probar: consola -> `localStorage.removeItem('record')`.

Añadir vidas (3 vidas; se pierde una al fallar)

Qué buscar: HTML de juego.html + variables globales + `atacar`.

ANTES

```
<p>Puntuación: <span id="puntuacion">0</span></p>

// juego.js
let puntuacion = 0;
```

DESPUÉS

```
<p>Puntuación: <span id="puntuacion">0</span></p>
<p>Vidas: <span id="vidas">3</span></p>

// juego.js
let puntuacion = 0;
let vidas = 3;

// en atacar(), tras calcular puntos:
if (puntos < 0) {
    vidas--;
    document.getElementById("vidas").textContent = vidas;
    if (vidas === 0) {
        document.getElementById("btnAtacar").disabled = true;
        document.getElementById("resultado").textContent = "¡Has perdido!";
        return;   // sale: no sigue comprobando victoria
    }
}
```

Por qué funciona: Una variable que cuenta, un `span` para verla, un `if` que la resta y otro que detecta el 0 (derrota). El `return` evita seguir ejecutando `atacar`.

Ojo: Las tres piezas: HTML con id, variable, lógica. Si falta una, no funciona.

Quitar el historial de ataques de la pantalla

Qué buscar: juego.html (párrafo del historial) y `atacar` (línea `getElementById('historial')`).

ANTES

```
<!-- juego.html -->
<p>Historial de ataques: <span id="historial">-</span></p>

// juego.js (atacar)
document.getElementById("historial").textContent = historial.join(", ");
```

DESPUÉS

```
<!-- juego.html: línea borrada -->

// juego.js: línea borrada (o comentada con //)
```

Por qué funciona: Si borras solo el HTML, `getElementById('historial')` devuelve `null` y `null.textContent` da error. **Siempre HTML y JS juntos.**

Ojo: NO borres `historial.push(...)`: lo necesitan `terminarMision` y `abrirAyuda`.

Añadir un botón Reiniciar

Qué buscar: juego.html (botón) + `iniciarJuego` (evento) + función nueva.

ANTES

```
<button type="button" id="btnAyuda">Ayuda</button>
```

DESPUÉS

```
<!-- HTML -->
<button type="button" id="btnReiniciar">Reiniciar</button>

// iniciarJuego():
document.getElementById("btnReiniciar").addEventListener("click", reiniciar);

// función nueva:
function reiniciar() {
    puntuacion = 0;
    historial = [];
    document.getElementById("puntuacion").textContent = 0;
    document.getElementById("intentos").textContent = 0;
    document.getElementById("historial").textContent = "-";
    document.getElementById("resultado").textContent = "";
    document.getElementById("btnAtacar").disabled = false;
}
```

Por qué funciona: Reiniciar = devolver al estado inicial TODO lo que `atacar` y `terminarMision` cambian: 2 variables, 4 textos y el botón desactivado.

Ojo: El olvido típico es `disabled = false`: tras ganar, el botón seguiría desactivado.

Añadir un límite de ataques (derrota por agotar intentos)

Qué buscar: Constante nueva + `atacar`.

ANTES

```
const OBJETIVO = 50;
```

DESPUÉS

```
const OBJETIVO = 50;
const MAX_ATAQUES = 15;

// al final de atacar():
if (puntuacion >= OBJETIVO) {
    terminarMision();
} else if (historial.length >= MAX_ATAQUES) {
    document.getElementById("btnAtacar").disabled = true;
    document.getElementById("resultado").textContent = "Misión fallida";
}
```

Por qué funciona: El `else if` solo se evalúa si no has ganado. Así el último ataque ganador no cuenta como derrota.

Cambiar los controles: atacar con la barra espaciadora

Qué buscar: `iniciarJuego`, justo tras enganchar los botones.

ANTES

```
document.getElementById("btnAtacar").addEventListener("click", atacar);
```

DESPUÉS

```
document.getElementById("btnAtacar").addEventListener("click", atacar);
document.addEventListener("keydown", function (e) {
    if (e.code === "Space") {
        atacar();
    }
});
```

Por qué funciona: El evento `keydown` entrega un objeto `e`; `e.code` indica la tecla. Se escucha en `document` porque ningún elemento tiene el foco.

Ojo: Si el botón Atacar está enfocado, la barra espaciadora también lo pulsaría: podría atacar dos veces. Para evitarlo añade `e.preventDefault();` dentro del `if`.

Que la ventana de ayuda reciba también el objetivo

Qué buscar: `abrirAyuda`, `ayuda.html`, `mostrarAyuda`.

ANTES

```
const url = "ayuda.html?nombre=" + encodeURIComponent(nombre) +
            "&puntuacion=" + puntuacion +
            "&ataques=" + historial.length;
```

DESPUÉS

```
// juego.js
const url = "ayuda.html?nombre=" + encodeURIComponent(nombre) +
            "&puntuacion=" + puntuacion +
            "&ataques=" + historial.length +
            "&faltan=" + (OBJETIVO - puntuacion);

<!-- ayuda.html -->
<p>Te faltan: <span id="faltan"></span> puntos</p>

// ayuda.js (en mostrarAyuda)
document.getElementById("faltan").textContent = parametros.get("faltan") || "-";
```

Por qué funciona: Los datos viajan en tres pasos: se añaden a la URL, se crea un hueco en el HTML y se lee el parámetro con el mismo nombre.

Ojo: Los paréntesis en `(OBJETIVO - puntuacion)` son obligatorios: sin ellos, `+` concatenaría textos.

4.4 Modificaciones difíciles

Ataque crítico: 10 % de probabilidad de doble daño

Qué buscar: `atacar`, justo después de calcular `puntos`.

ANTES

```
const puntos = Math.floor(Math.random() * 16) - 5;
```

DESPUÉS

```
let puntos = Math.floor(Math.random() * 16) - 5;   // let: ahora cambia
let critico = false;
if (puntos > 0 && Math.random() < 0.1) {
    puntos = puntos * 2;
    critico = true;
}
// ... y en el mensaje positivo:
resultado.textContent = (critico ? "¡CRÍTICO! +" : "¡Golpe! +") + puntos + " puntos";
```

Por qué funciona: `Math.random() < 0.1` es verdadero el 10 % de las veces. Hay que pasar `puntos` de `const` a `let` porque se reasigna.

Ojo: Cambiar `const` por `let` es obligatorio: reasignar una `const` da `TypeError`.

Mostrar los 3 mejores ataques al ganar (ordenar un array)

Qué buscar: `terminarMision`.

ANTES

```
document.getElementById("resultado").textContent =
    "¡Misión completada! Mejor ataque: " + mejor + " | Media: " + media;
```

DESPUÉS

```
// copia del array (para no desordenar el historial real)
const ordenados = [...historial].sort(function (a, b) { return b - a; });
const top3 = ordenados.slice(0, 3);
document.getElementById("resultado").textContent =
    "¡Misión completada! Top 3: " + top3.join(", ") + " | Media: " + media;
```

Por qué funciona: `sort` sin función ordena como TEXTO ('10' antes que '2'); con `(a, b) => b - a` ordena números de mayor a menor. `slice(0, 3)` toma las 3 primeras posiciones. `[...historial]` copia para no alterar el original.

Contar ataques fallidos (usando un bucle)

Qué buscar: `terminarMision`, aprovechando el `for` que ya existe.

ANTES

```
let suma = 0;
for (let i = 0; i < historial.length; i++) {
    suma += historial[i];
}
```

DESPUÉS

```
let suma = 0;
let fallos = 0;
for (let i = 0; i < historial.length; i++) {
    suma += historial[i];
    if (historial[i] < 0) {
        fallos++;
    }
}
```

Por qué funciona: En el mismo recorrido se acumula la suma y se cuenta el fallo; un contador empieza en 0 y se incrementa con `++`.

5. Errores habituales

5.1 Errores al modificar el código

Error	Ejemplo en tu juego	Cómo evitarlo
Borrar HTML y olvidar el JS	Quitas `#historial` y `atacar` sigue escribiendo en él -> `Cannot set properties of null`	Busca el id con Ctrl+F en los .js antes de borrar
Añadir JS sin id en el HTML	`getElementById('vidas')` devuelve `null`	HTML primero
Cambiar un id en un solo sitio	`btnAtacar` -> `btnAtaque` en HTML pero no en JS	Cambia ambos y recarga
Cambiar `const` y reasignar	`const puntos` y luego `puntos = puntos*2`	Usa `let`
Poner paréntesis en el evento	`addEventListener('click', atacar())` ejecuta ya	Sin paréntesis: `atacar`
Variable declarada dentro de función	`let vidas = 3;` dentro de `atacar` se reinicia cada clic	Declárala fuera (global)
Olvidar `Number()`	Comparar `'9' < '10'` como texto da false	Convierte con `Number()`
Sumar texto en vez de número	`parametros.get('puntuacion') + 5` da '205'	`Number(...) + 5`
`=` en vez de `===`	`if (edad = 18)` asigna	Comparar con `===`
Olvidar `return` tras un error	En `validarPerfil` entraría igualmente	Un `return` por cada filtro
Cambiar la clave del storage en un sitio	`setItem('record')` y `getItem('records')`	Mismo texto exacto

5.2 Partes que NO se deben tocar sin modificar otras

Si tocas...	...revisa también
`OBJETIVO`	Pista de ayuda.js ('50 puntos')
Rango de puntos en `atacar`	Pista 'entre -5 y 10' en ayuda.js, y el comentario
Un id del HTML	Todos los `getElementById` de ese id
Un parámetro de la URL en `abrirAyuda`	`parametros.get(...)` de ayuda.js
La clave 'nombre' / 'fechaNacimiento' de sessionStorage	`validarPerfil` (guarda), `iniciarJuego` y `mostrarPanel` (leen)
La clave 'record' de localStorage	`mostrarRecord` y `guardarRecord`
`historial`	`atacar`, `terminarMision`, `abrirAyuda`
`calcularEdad`	Está duplicada en script.js y juego.js: cambia ambas
El nombre de la cookie `ultimoJugador`	`guardarCookie` en `validarPerfil` y `leerCookie` en DOMContentLoaded

5.3 Errores de sintaxis típicos

- Falta una llave `}` o paréntesis `)`: todo el archivo deja de funcionar.
- Falta cerrar comillas en un texto.
- Falta `+` al concatenar: `"Puntos: " puntos` es un error.
- Falta `;` o coma entre elementos de un array / parámetros de una función.
- Mayúsculas/minúsculas: `getElementbyId` no existe, es `getElementById`.
- `textcontent` en vez de `textContent`: no da error pero no hace nada (crea una propiedad nueva).
- `else if` escrito como `elseif` o `else-if`.
- Usar comillas tipográficas (“ ”) copiadas de Word: usa comillas rectas (" ').
- `for (let i = 0; i < historial.length, i++)`: en el `for` se separa con `;`, no con coma.

5.4 Cómo detectar errores en la consola

- Abre el navegador y pulsa **F12** -> pestaña **Console**. Los errores salen en rojo con el archivo y la línea (por ejemplo `juego.js:43`).
- `Uncaught ReferenceError: x is not defined`: variable o función mal escrita o fuera de alcance.
- `Cannot read properties of null (reading 'textContent')` / `Cannot set properties of null`: un `getElementById` no encontró el id (id mal escrito o falta en el HTML).
- `Uncaught SyntaxError`: falta o sobra un símbolo; el archivo entero deja de ejecutarse, por eso 'no funciona nada'.
- `TypeError: Assignment to constant variable`: reasignaste una `const`.
- Usa `console.log(variable)` para ver valores: por ejemplo `console.log(puntos, puntuacion)` dentro de `atacar`.
- En la pestaña **Application** (Aplicación) puedes ver cookies, sessionStorage y localStorage y borrarlos.
- Recarga forzada con **Ctrl+F5** para que no use una versión antigua en caché del JS.

5.5 Cómo comprobar que un cambio funciona

- Prueba el caso normal y también los límites: nombre vacío, fecha futura, menor de edad, 0 puntos, justo 50.
- Para probar la victoria rápido: pon temporalmente `OBJETIVO = 5` y vuelve a ponerlo.
- Para probar `index` sin cookie: borra la cookie en Application. Para probar el récord: borra `record` en localStorage.
- Comprueba que lo que NO tocaste sigue funcionando (los 4 botones, el panel, la ayuda).
- Abre la consola y confirma que no queda ningún error rojo.

6. Diccionario de JavaScript

Solo conceptos que aparecen en tu código, cada uno con un ejemplo sencillo y dónde se usa.

Concepto	Qué es / ejemplo	Dónde en tu código
`const`	Constante, no se puede reasignar. `const OBJETIVO = 50;`	OBJETIVO, EDAD_MINIMA, pistas, variables locales
`let`	Variable que cambia. `let suma = 0; suma += 3;`	puntuacion, historial, suma, edad, i
`function`	Bloque reutilizable. `function saludar(n) { return 'Hola ' + n; }`	Todas las funciones
función anónima	Función sin nombre pasada como valor. `function () {...}`	DOMContentLoaded de script.js
parámetros	Datos de entrada: `calcularEdad(fecha)`	calcularEdad, guardarCookie(nombre, valor, dias)
`return`	Devuelve un valor y/o sale de la función	calcularEdad, leerCookie, validarPerfil (salir)
`if / else if / else`	Decisión. `if (x > 0) {...} else {...}`	atacar, validarPerfil, aplicarEstilosPanel
`for`	Repetir. `for (let i = 0; i < n; i++) {...}`	terminarMision, leerCookie
Array `[]`	Lista. `const a = [1, 2]; a[0]`	historial, pistas, cookies
`push`	Añade al final del array	`historial.push(puntos)`
`.length`	Tamaño de array o texto	`historial.length`, `pistas.length`
`join(', ')`	Une un array en texto	mostrar historial
`split('; ')`	Parte un texto en array	leerCookie
spread `...`	Esparce un array. `Math.max(...[3,9,2])` = 9	terminarMision
`Math.random()`	Decimal de 0 (incluido) a 1 (excluido)	atacar, mostrarAyuda
`Math.floor`	Redondea hacia abajo. 7.9 -> 7	atacar, mostrarAyuda
`Math.round`	Redondea al más cercano. 2.5 -> 3	terminarMision (media)
`Math.max`	El mayor. `Math.max(0, -3)` = 0	atacar, terminarMision
`Number()`	Convierte texto a número	guardarRecord
`isNaN()`	True si no es un número válido	validarPerfil
Operadores `+ - * /`	Aritmética; `+` también concatena textos	Todo el código
`++ -- +=`	`edad--` resta 1; `suma += x` equivale a `suma = suma + x`	calcularEdad, terminarMision
`=== !== < > >= <=`	Comparación. `===` compara valor y tipo	Condiciones
`&& ||`	Y lógico / O lógico. Con `||` sirve de valor por defecto	calcularEdad (&&, ||), ayuda.js (`|| '-'`)
`null`	'Sin valor'. Lo devuelven `getItem` y `params.get` si no existe	iniciarJuego, mostrarRecord
Concatenación	`'Hola ' + nombre`	Mensajes
String: `trim`	Quita espacios de los extremos	validarPerfil
String: `charAt(0)`	Carácter en esa posición	mostrarPanel, aplicarEstilosPanel
String: `slice(1)`	Desde esa posición hasta el final	mostrarPanel

Concepto	Qué es / ejemplo	Dónde en tu código
`toUpperCase / toLowerCase`	Mayúsculas / minúsculas	mostrarPanel
String: `includes`	Comprueba si contiene texto	aplicarEstilosPanel
`new Date()`	Crea fecha. Sin parámetro = ahora; con texto 'AAAA-MM-DD' = esa fecha	calcularEdad, validarPerfil, guardarCookie
Date: `getFullYear`, `getMonth`, `getDate`	Año, mes (0-11), día del mes	calcularEdad
Date: `getTime()`	Milisegundos; es NaN si la fecha no es válida	validarPerfil
Date: `setDate`, `toUTCString`	Cambia el día / formato para cookies	guardarCookie
`document.getElementById`	Obtiene un elemento por su id	En todos los .js
`.textContent`	Lee o escribe el texto de un elemento	Casi todas las funciones
`.value`	Valor de un input	script.js
`.disabled`	Desactiva un botón (true/false)	terminarMision
`.style.propiedad`	Cambia CSS desde JS (camelCase)	aplicarEstilosPanel
`addEventListener`	Asocia una función a un evento	Todos los botones y DOMContentLoaded
`click`, `DOMContentLoaded`	Eventos: clic / HTML cargado	-
`defer`	Ejecutar el script tras cargar el HTML	Los 3 `<script>` de los HTML
`window.location.href`	Lee o cambia la dirección actual (navega)	validarPerfil, iniciarJuego, salir
`window.location.search`	La parte de la URL desde `?`	mostrarAyuda
`URLSearchParams`	Lee parámetros de la URL con `get`	mostrarAyuda
`window.open`	Abre ventana nueva (url, nombre, características)	abrirAyuda
`window.close`	Cierra la ventana	cerrarVentana
`encodeURIComponent / decodeURIComponent`	Codifica / decodifica caracteres especiales	abrirAyuda, cookies
`document.cookie`	Texto con todas las cookies; asignarle una crea/actualiza esa cookie	script.js
`sessionStorage`	`setItem`, `getItem`, `clear` (solo pestaña actual)	script.js, juego.js
`localStorage`	`setItem`, `getItem` (permanente)	mostrarRecord, guardarRecord
Comentarios	`// una línea` y `/* varias */`	Por todo

Conceptos que NO aparecen pero conviene conocer

El examen puede pedirte usar algo parecido. Resumen rápido:

Concepto	Para qué	Ejemplo
`setTimeout(fn, ms)`	Ejecuta una función UNA vez tras un tiempo	`setTimeout(terminarMision, 30000);`
`setInterval(fn, ms)`	Ejecuta una función cada X ms; se para con `clearInterval(id)`	`const id = setInterval(tick, 1000);`

Concepto	Para qué	Ejemplo
`querySelector('#id')`	Como `getElementById`, pero con selectores CSS	`document.querySelector('#panel')`
`classList.add/remove/toggle`	Añade o quita clases CSS	`panel.classList.add('adulto')`
`while`	Bucle mientras una condición sea cierta	`while (n > 0) { n--; }`
Objetos `{}`	Datos con propiedades	`const j = { nombre: 'Ana', vidas: 3 }; j.vidas`
Función flecha	Forma corta de función	`(a, b) => a + b`
Operador ternario	`if` en una línea	`x > 0 ? 'positivo' : 'no'`
`removeItem`	Borra una clave del storage	`localStorage.removeItem('record')`

```
// setInterval: cuenta atrás de 30 s (ejemplo)
let tiempo = 30;
const id = setInterval(function () {
    tiempo--;
    document.getElementById("tiempo").textContent = tiempo;
    if (tiempo === 0) {
        clearInterval(id);
        document.getElementById("btnAtacar").disabled = true;
    }
}, 1000);
```

7. Ejercicios tipo examen

18 ejercicios sobre tu código real, de fácil a difícil. **Intenta resolverlos tú primero** en una copia del proyecto y solo después mira la solución.

Ejercicio 1 - Cambiar el objetivo de la misión [Fácil]

Te piden: Que la misión se complete con 80 puntos en lugar de 50.

Necesitas saber: Constantes (`const`).

Pasos:

- Buscar `OBJETIVO` en juego.js.
- Cambiar el número.
- (Opcional) actualizar la pista de ayuda.js.

SOLUCIÓN

```
const OBJETIVO = 80;

// ayuda.js (coherencia)
"Necesitas 80 puntos para completar la misión.",
```

Por qué se hace así: `OBJETIVO` se usa en `iniciarJuego` (pantalla) y en `atacar` (victoria). Un único cambio afecta a todo.

Ejercicio 2 - Edad mínima y cookie [Fácil]

Te piden: Exigir 16 años y recordar al jugador durante 30 días.

Necesitas saber: Constantes, parámetros de función.

Pasos:

- En script.js cambiar `EDAD_MINIMA`.
- En `validarPerfil` cambiar el 7 por 30 en `guardarCookie`.

SOLUCIÓN

```
const EDAD_MINIMA = 16;
...
guardarCookie("ultimoJugador", nombre, 30);
```

Por qué se hace así: La edad se compara en el filtro 4; los días son el tercer parámetro de `guardarCookie`.

Ejercicio 3 - Nuevo rango de ataque [Fácil]

Te piden: Cada ataque debe dar entre 0 y 20 puntos (sin negativos).

Necesitas saber: `Math.random`, `Math.floor`.

Pasos:

- Ir a `atacar`.
- Hay 21 valores posibles (0..20): multiplicar por 21.
- Quitar el `- 5`.
- Opcional: borrar el mensaje de 'Fallaste' o dejarlo (ya no se ejecutará).

SOLUCIÓN

```
const puntos = Math.floor(Math.random() * 21);
```

Por qué se hace así: `random()*21` va de 0 a 20.999; `floor` lo deja en 0..20. Sin la resta no hay negativos.

Ejercicio 4 - Cambiar colores del panel [Fácil]

Te piden: Fondo rojo (#b22222) si el nombre empieza por vocal, y texto amarillo si el jugador es menor de 21.

Necesitas saber: `style`, if/else.

Pasos:

- Ir a `aplicarEstilosPanel`.
- Cambiar el color del bloque `if` de vocales.
- Cambiar 18 por 21 en `if (edad < 18)`.

SOLUCIÓN

```
if ("aeiouáéíóú".includes(inicial)) {
    panel.style.backgroundColor = "#b22222";
} else {
    panel.style.backgroundColor = "#7a3e8e";
}
if (edad < 21) {
    panel.style.color = "#ffe066";
} else {
    panel.style.color = "#ffffff";
}
```

Por qué se hace así: Los colores son strings asignados a propiedades de `style`; el límite de edad es la condición del `if`.

Ejercicio 5 - Nombre en mayúsculas y su longitud [Fácil]

Te piden: Mostrar en el panel el nombre completo en mayúsculas y cuántas letras tiene.

Necesitas saber: Métodos de String, DOM.

Pasos:

- En juego.html añadir `<p>Letras: <span id="letras"></span></p>` dentro del panel.
- En `mostrarPanel` usar `toUpperCase()` y `.length`.

SOLUCIÓN

```
// HTML (dentro de #panel)
<p>Letras: <span id="letras"></span></p>

// mostrarPanel:
const nombreFormateado = nombre.toUpperCase();
...
document.getElementById("letras").textContent = nombre.length;
```

Por qué se hace así: `toUpperCase()` convierte todo el texto; `length` cuenta caracteres. El HTML nuevo necesita su id.

Ejercicio 6 - Mensajes de error distintos [Fácil-Media]

Te piden: En vez de siempre 'Acceso denegado', mostrar la causa concreta.

Necesitas saber: if, textContent, `return`.

Pasos:

- Ir a `validarPerfil`.
- En cada filtro cambiar el texto según el motivo.
- Mantener un `return` en cada uno.

SOLUCIÓN

```
if (nombre === "") {
    mensaje.textContent = "Acceso denegado: escribe tu nombre";
    return;
}
if (fechaTexto === "" || isNaN(fecha.getTime())) {
    mensaje.textContent = "Acceso denegado: fecha no válida";
    return;
}
if (fecha >= new Date()) {
    mensaje.textContent = "Acceso denegado: la fecha debe ser anterior a hoy";
    return;
}
if (calcularEdad(fecha) < EDAD_MINIMA) {
    mensaje.textContent = "Acceso denegado: edad mínima " + EDAD_MINIMA;
    return;
}
```

Por qué se hace así: Cada `if` es un filtro independiente; el `return` evita que continúe. Se concatena `EDAD_MINIMA` para que el texto no se desfase.

Ejercicio 7 - Que el récord funcione [Media]

Te piden: El récord (menos ataques para ganar) debe guardarse y mostrarse al volver a jugar.

Necesitas saber: Llamar funciones, localStorage, `historial.length`.

Pasos:

- Detectar que `guardarRecord` no se llama en ningún sitio.
- Llamarla en `terminarMision` pasando el número de ataques.

SOLUCIÓN

```
function terminarMision() {
    // ... código existente ...
    guardarRecord(historial.length);
}
```

Por qué se hace así: `guardarRecord` ya contiene la lógica (primer récord o mejora). Solo faltaba invocarla. `mostrarRecord` lo lee al cargar.

Ejercicio 8 - Mostrar cuántos puntos faltan [Media]

Te piden: Mostrar 'Faltan X puntos' tras cada ataque.

Necesitas saber: Aritmética, DOM.

Pasos:

- Añadir `<p>Faltan: <span id="faltan">50</span></p>` al HTML.
- En `atacar`, tras actualizar la puntuación, escribir `OBJETIVO - puntuacion`.
- Evitar números negativos con `Math.max(0, ...)`.

SOLUCIÓN

```
// juego.html
<p>Faltan: <span id="faltan"></span></p>

// atacar(), junto a las demás actualizaciones:
document.getElementById("faltan").textContent = Math.max(0, OBJETIVO - puntuacion);
```

Por qué se hace así: Al pasar de 50, `OBJETIVO - puntuacion` sería negativo; `Math.max(0, x)` lo corrige.

Ejercicio 9 - Contar ataques fallidos y mostrarlos al ganar [Media]

Te piden: En el mensaje final indicar cuántos ataques tuvieron puntos negativos.

Necesitas saber: Bucle `for`, contador, if.

Pasos:

- En `terminarMision` crear `let fallos = 0;`.
- Dentro del `for` existente, `if (historial[i] < 0) fallos++;`.
- Añadirlo al mensaje.

SOLUCIÓN

```
let suma = 0;
let fallos = 0;
for (let i = 0; i < historial.length; i++) {
    suma += historial[i];
    if (historial[i] < 0) {
        fallos++;
    }
}
...
document.getElementById("resultado").textContent =
    "¡Misión completada! Mejor ataque: " + mejor + " | Media: " + media + " | Fallos: " + fallos;
```

Por qué se hace así: El bucle ya recorre todo el historial; se aprovecha para contar. El contador debe declararse FUERA del bucle.

Ejercicio 10 - Botón Reiniciar [Media]

Te piden: Añadir un botón que reinicie la partida sin recargar.

Necesitas saber: Eventos, reasignar variables, `disabled`.

Pasos:

- Botón con id en juego.html.
- Enlazarlo en `iniciarJuego`.
- Crear `reiniciar()` que reinicie variables Y pantalla.

SOLUCIÓN

```
<button type="button" id="btnReiniciar">Reiniciar</button>

// iniciarJuego:
document.getElementById("btnReiniciar").addEventListener("click", reiniciar);

function reiniciar() {
    puntuacion = 0;
    historial = [];
    document.getElementById("puntuacion").textContent = 0;
    document.getElementById("intentos").textContent = 0;
    document.getElementById("historial").textContent = "-";
    document.getElementById("resultado").textContent = "";
    document.getElementById("btnAtacar").disabled = false;
}
```

Por qué se hace así: Hay que restaurar el estado en memoria (variables) y el estado visible (textos y botón).

Ejercicio 11 - Límite de ataques (derrota) [Media]

Te piden: Si tras 15 ataques no se ha ganado, 'Misión fallida' y desactivar Atacar.

Necesitas saber: Constantes, `else if`, `historial.length`.

Pasos:

- Constante `MAX_ATAQUES = 15`.
- En `atacar`, convertir la comprobación final en if / else if.

SOLUCIÓN

```
const MAX_ATAQUES = 15;

// final de atacar():
if (puntuacion >= OBJETIVO) {
    terminarMision();
} else if (historial.length >= MAX_ATAQUES) {
    document.getElementById("btnAtacar").disabled = true;
    document.getElementById("resultado").textContent = "Misión fallida";
}
```

Por qué se hace así: El `else if` solo se evalúa si no ganó, y `historial.length` ya es el nº de ataques realizados.

Ejercicio 12 - Más información en la ayuda [Media]

Te piden: La ventana de ayuda debe mostrar también los puntos que faltan para ganar.

Necesitas saber: Parámetros URL, `URLSearchParams`.

Pasos:

- En `abrirAyuda` añadir `&faltan=` a la URL.
- En ayuda.html crear un `span` con id.
- En `mostrarAyuda` leer el parámetro.

SOLUCIÓN

```
// juego.js (abrirAyuda)
"&faltan=" + Math.max(0, OBJETIVO - puntuacion)

// ayuda.html
<p>Faltan: <span id="faltan"></span></p>

// ayuda.js (mostrarAyuda)
document.getElementById("faltan").textContent = parametros.get("faltan") || "-";
```

Por qué se hace así: Los datos pasan por la URL como texto y ayuda.js los lee con el mismo nombre de parámetro.

Ejercicio 13 - Pista según el progreso [Media-Difícil]

Te piden: En la ayuda, si la puntuación es >= 30 mostrar siempre '¡Casi lo tienes!' en lugar de una pista aleatoria.

Necesitas saber: Parámetros llegan como texto -> `Number()`; if/else.

Pasos:

- Leer `puntuacion` con `Number(parametros.get('puntuacion'))`.
- Si >= 30, texto fijo. Si no, pista aleatoria.

SOLUCIÓN

```
const puntos = Number(parametros.get("puntuacion"));
if (puntos >= 30) {
    document.getElementById("pista").textContent = "¡Casi lo tienes!";
} else {
    const posicion = Math.floor(Math.random() * pistas.length);
    document.getElementById("pista").textContent = pistas[posicion];
}
```

Por qué se hace así: El valor es texto, aunque parezca número. Si no existe el parámetro, `Number(null)` da 0, así que no rompe.

Ejercicio 14 - Ataque crítico [Difícil]

Te piden: 10 % de las veces con puntos positivos, el ataque se duplica y el mensaje dice '¡CRÍTICO!'.

Necesitas saber: `let` vs `const`, `Math.random`, probabilidad, ternario.

Pasos:

- Cambiar `const puntos` a `let puntos`.
- Probabilidad: `Math.random() < 0.1`.
- Marcar con una variable `critico` y usarla en el mensaje.

SOLUCIÓN

```
let puntos = Math.floor(Math.random() * 16) - 5;
let critico = false;
if (puntos > 0 && Math.random() < 0.1) {
    puntos = puntos * 2;
    critico = true;
}
// ...
if (puntos > 0) {
    resultado.textContent = (critico ? "¡CRÍTICO! +" : "¡Golpe! +") + puntos + " puntos";
}
```

Por qué se hace así: `random()` está entre 0 y 1, así que `< 0.1` ocurre el 10 % de las veces. La doble condición `&&` exige puntos positivos Y suerte.

Ejercicio 15 - Récord de la mejor puntuación media [Difícil]

Te piden: Guardar en localStorage la mejor MEDIA de ataque (la más alta) y mostrarla al terminar.

Necesitas saber: localStorage, `Number`, comparar, texto.

Pasos:

- En `terminarMision`, después de calcular `media`.
- Leer `mejorMedia` y convertir con `Number`.
- Guardar si no existe o si la nueva es mayor.
- Mostrarla en el mensaje.

SOLUCIÓN

```
const guardada = localStorage.getItem("mejorMedia");
if (guardada === null || media > Number(guardada)) {
    localStorage.setItem("mejorMedia", media);
}
document.getElementById("resultado").textContent =
    "¡Misión completada! Media: " + media +
    " | Mejor media: " + localStorage.getItem("mejorMedia");
```

Por qué se hace así: Es el mismo patrón que `guardarRecord`, pero la condición invierte la comparación (aquí MÁS es mejor).

Ejercicio 16 - Top 3 de ataques [Difícil]

Te piden: Al ganar, mostrar los 3 ataques más altos de la partida.

Necesitas saber: `sort` numérico, `slice`, copiar arrays.

Pasos:

- Copiar el array con `[...historial]`.
- Ordenar de mayor a menor con `(a, b) => b - a`.
- Tomar 3 con `slice(0, 3)`.
- Unir con `join`.

SOLUCIÓN

```
const ordenados = [...historial].sort(function (a, b) { return b - a; });
const top3 = ordenados.slice(0, 3).join(", ");
document.getElementById("resultado").textContent =
    "¡Misión completada! Top 3: " + top3 + " | Media: " + media;
```

Por qué se hace así: Sin la función de comparación, `sort` ordena como texto. Se copia el array para no desordenar el historial real.

Ejercicio 17 - Límite de tiempo con temporizador [Difícil]

Te piden: La partida dura 30 segundos. Mostrar la cuenta atrás; al llegar a 0 sin ganar, desactivar Atacar y mostrar 'Tiempo agotado'.

Necesitas saber: `setInterval`, `clearInterval`, estado compartido.

Pasos:

- HTML: `<p>Tiempo: <span id="tiempo">30</span></p>`.
- Variables globales `tiempo` y `idTemporizador`.
- En `iniciarJuego` arrancar el intervalo cada 1000 ms.
- Parar el intervalo al ganar (`terminarMision`) para que no siga contando.

SOLUCIÓN

```
// juego.html
<p>Tiempo: <span id="tiempo">30</span></p>

// juego.js
let tiempo = 30;
let idTemporizador;

// dentro de iniciarJuego():
idTemporizador = setInterval(function () {
    tiempo--;
    document.getElementById("tiempo").textContent = tiempo;
    if (tiempo === 0) {
        clearInterval(idTemporizador);
        document.getElementById("btnAtacar").disabled = true;
        document.getElementById("resultado").textContent = "Tiempo agotado";
    }
}, 1000);

// dentro de terminarMision():
clearInterval(idTemporizador);
```

Por qué se hace así: `setInterval` repite una función cada X ms y devuelve un identificador; hay que guardarlo para poder pararlo con `clearInterval` al ganar o perder.

Ejercicio 18 - Contador de visitas con cookie [Difícil]

Te piden: En index.html, mostrar 'Visitas: N' con un contador guardado en una cookie de 365 días.

Necesitas saber: Cookies, `Number`, funciones existentes.

Pasos:

- HTML: `<p id="visitas"></p>`.
- En DOMContentLoaded leer la cookie con `leerCookie`.
- Convertirla con `Number`; si es '' da 0.
- Sumar 1, guardar y mostrar.

SOLUCIÓN

```
// index.html
<p id="visitas"></p>

// script.js, dentro de DOMContentLoaded:
let visitas = Number(leerCookie("visitas"));   // "" -> 0
visitas++;
guardarCookie("visitas", visitas, 365);
document.getElementById("visitas").textContent = "Visitas: " + visitas;
```

Por qué se hace así: `Number('')` devuelve 0, así que la primera vez cuenta 1. Se reutilizan `leerCookie` y `guardarCookie` sin reescribirlas.

8. Resumen final para estudiar

8.1 Esquema general

```
index.html + script.js
  Entrar -> validarPerfil()
    |- nombre vacío / fecha inválida / fecha futura / edad < 12  -> "Acceso denegado"
    '- OK -> sessionStorage(nombre, fechaNacimiento) + cookie(ultimoJugador, 7d) -> juego.html

juego.html + juego.js  (iniciarJuego)
  |- sin nombre en sessionStorage -> vuelve a index.html
  |- pinta: saludo, OBJETIVO, mostrarRecord(), mostrarPanel()
  |-  Atacar -> atacar(): puntos(-5..10), puntuacion=max(0,..), historial.push, textos
  |              si puntuacion >= 50 -> terminarMision(): media, mejor, desactiva Atacar
  |-  Ayuda  -> abrirAyuda(): window.open("ayuda.html?nombre=..&puntuacion=..&ataques=..")
  '-  Salir  -> sessionStorage.clear() -> index.html

ayuda.html + ayuda.js  (mostrarAyuda)
  lee parámetros de la URL + pista aleatoria; Cerrar -> window.close()
```

8.2 Funciones más importantes

Función	Archivo	Utilidad
`validarPerfil`	script.js	Puerta de entrada: 4 validaciones + guardar datos + saltar al juego
`calcularEdad`	script.js / juego.js	Edad en años cumplidos con Date
`guardarCookie` / `leerCookie`	script.js	Crear y leer cookies
`iniciarJuego`	juego.js	Prepara pantalla y eventos
`atacar`	juego.js	Mecánica central: aleatorio, puntuación, array, mensajes, victoria
`terminarMision`	juego.js	Fin de partida: media, mejor, desactivar botón
`mostrarPanel` / `aplicarEstilosPanel`	juego.js	Panel con String, Date y estilos
`abrirAyuda`	juego.js	Ventana nueva con datos por URL
`mostrarRecord` / `guardarRecord`	juego.js	localStorage del récord (guardar sin usar)
`salir`	juego.js	Limpia sesión y vuelve
`mostrarAyuda` / `cerrarVentana`	ayuda.js	Leer URL, pista aleatoria, cerrar

8.3 Variables, claves e ids que debes saber de memoria

Qué	Valores
Constantes	`EDAD_MINIMA = 12` (script.js); `OBJETIVO = 50` (juego.js)
Estado del juego	`puntuacion` (número), `historial` (array)
Array de pistas	`pistas` (4 textos, ayuda.js)
sessionStorage	`nombre`, `fechaNacimiento`
localStorage	`record`
Cookie	`ultimoJugador` (7 días)
Parámetros URL	`nombre`, `puntuacion`, `ataques`

Qué	Valores
Ids de index	`formPerfil`, `nombre`, `fechaNacimiento`, `btnEntrar`, `mensaje`
Ids de juego	`saludo`, `panel`, `nombreFormateado`, `edad`, `objetivo`, `puntuacion`, `intentos`, `record`, `btnAtacar`, `btnAyuda`, `resultado`, `historial`, `btnSalir`
Ids de ayuda	`nombre`, `puntuacion`, `ataques`, `pista`, `btnCerrar`
Fórmula de ataque	`Math.floor(Math.random() * 16) - 5` -> -5..10
Fórmula edad	año actual - año nacimiento, menos 1 si aún no cumplió

8.4 Zonas donde se hacen los cambios más habituales

Si te piden...	Ve a...
Cambiar dificultad	Línea 2 de juego.js (`OBJETIVO`) y `atacar` (rango)
Cambiar mensajes	`atacar` (if/else if/else), `terminarMision`, `validarPerfil`
Cambiar colores	`aplicarEstilosPanel` y estilos.css
Cambiar el panel	`mostrarPanel` + `#panel` en juego.html
Nuevo botón	HTML + `iniciarJuego` (addEventListener) + función nueva
Algo con datos guardados	script.js (cookies/sessionStorage) o juego.js (localStorage)
La ventana de ayuda	`abrirAyuda` + ayuda.js + ayuda.html
Validaciones del formulario	`validarPerfil`

8.5 Lista de imprescindibles para aprobar

- Saber qué archivo carga a qué script y qué función arranca cada página (`DOMContentLoaded`).
- Escribir de memoria `addEventListener('click', funcion)` (sin paréntesis) y `document.getElementById('id').textContent = valor`.
- Entender `Math.floor(Math.random() * N)` y cómo ajustarlo a cualquier rango.
- Saber la diferencia `const`/`let` y cuándo usar cada una.
- Saber que `getItem` devuelve SIEMPRE texto o `null` y que se convierte con `Number()`.
- Distinguir cookie (días), sessionStorage (pestaña) y localStorage (permanente) y sus funciones.
- Pasar datos por URL (`?a=1&b=2`) y leerlos con `URLSearchParams`.
- Recorrer un array con `for`, `push`, `join`, `length`, `Math.max(...array)`.
- Método String: `trim`, `charAt`, `slice`, `toUpperCase`, `toLowerCase`, `includes`.
- Date: `new Date()`, `getFullYear`, `getMonth` (0-11), `getDate`, `setDate`, `toUTCString`.
- Toda cosa nueva en pantalla: HTML con id + JS + (opcional) CSS.
- Nunca borrar HTML sin borrar el JS que lo usa (y viceversa).
- Abrir la consola (F12) y saber interpretar `null`, `not defined` y `SyntaxError`.
- Recordar el fallo escondido: `guardarRecord` no se llama en ningún sitio.

Recomendación final de estudio
Haz una copia del proyecto y, sin mirar este manual, resuelve los ejercicios 1-6 (15 minutos), 7-12 (30 minutos) y 13-18 (45 minutos). Si un ejercicio te bloquea, vuelve a la ficha de la función correspondiente en la sección 2.